山东大学 计算机科学与技术 学院

数据结构与算法 课程实验报告

学号：202200130077	姓名：薛润泽	班级：人工智能
实验题目：A:图论基础		
实验学时：2	实验日期：2023.12.6	

实验目的：

创建无向图类，存储结构使用邻接链表，提供操作：插入一条边，删除一条边，BFS，DFS。

软件开发工具：

DEV C++

1. 实验内容

创建无向图类，存储结构使用邻接链表，提供操作：插入一条边，删除一条边，BFS，DFS。

2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

所需数据结构与算法：邻接链表，递归，队列，深度优先搜索，广度优先搜索

整体思路描述：

（1）建图，通过使用函数 push()和函数 deleteElement 完成建图

void push(Node* a, int b);建立点 a 与点 b 的边。通过遍历 a 的邻接链表，找到 b 应该放入的顺序，new Node(b)后将 b 节点插入，同时维护邻接链表的有序性。

void deleteElement(Node* cnt, int b);遍历 cnt 的邻接链表，找到相应删除点，进行点的删除。

（2）使用路径压缩的并查集查询连通分量：

int findacs(int son);递归查找 son 的祖宗节点，当 son == parent[son]的时候返回。在递归返回的过程中更新访问到的 son 的 parent[son]。

void merge(int a, int b);合并 a,b 两个点所在集合。我们选择将 parent 更大的祖宗更新为另一个更小的祖宗。例如 parent[a] < parent[b],我们更新 parent[b]=parent[a];

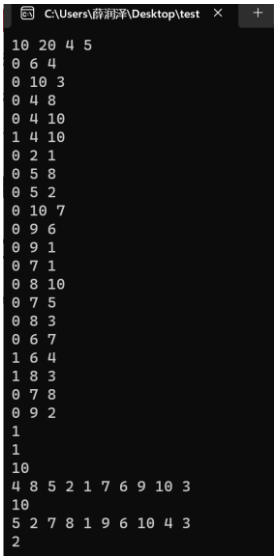
void update();使用 merge 来更新图中所有点的集合情况。

void Dividenum();用来输出连通分量个数和连通子图中最小的序号。

（3） 深度优先搜索：void dfs()；通过递归实现深度优先搜索。过程中使用一个 bool 类型数组，记录点是否被访问，如果点已经被访问过，则不再访问。

（4）广度优先搜索：void bfs()；使用队列进行广度优先搜索。同时记录广度优先搜索的层数。用于输出 s 点到 t 点距离。

3. 测试结果（测试输入，测试输出）



4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

在写代码的过程中，我遇到了输出结果与样例结果正确，但是在 OJ 中提交全错的情况。错误情况为 1 个 RE，8 个 WA。通过不断的查找错误，检查代码，我发现我在实现并查集操作 findacs()中缺少了 return。没有写 return 在本地机中没有显示错误，但是评测机却是错误的。现在想来，错误情况实际应该是 9 个 RE。我通过添加了 return，成功完成评测。

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

```
#include<iostream>
//循环队列开始
template<typename T>
class Queue{//循环队列，实现 push，pop，top 等功能，先入先出
public:
    Queue(int n){//构造函数
        qBack = 0;
        qFront = 0;
        arrayLength = n + 1;
        element = new T[n + 1];//申请 n+1 个空间，因为循环队列数组中要空一个
    }
    ~Queue(){
        delete []element;
    }
    int Size(){//判断队列中元素数量
        return   (arrayLength + qBack - qFront)%arrayLength;//模 arrayLength 保证 qBack 在 qFront 右方时计算结果不超出 arrayLength，
    }
    bool empty(){//判断队列是否为空
        return qBack == qFront;//当 qBack = qFront 时为空，(qBack + 1)%arrayLength = qFront 时为满
    }
    void push(T a){
        if((qBack + 1)%arrayLength == qFront){//判断队列是否满
            std::cout<<"FULL QUEUE"<<'\n';
```

```
            return;
        }
        qBack = (qBack + 1)%arrayLength;
        element[qBack] = a;
    }
    T top(){
        if(this->empty()) std::cout<<"empty"<<'\n';
        return element[(qFront + 1)%arrayLength];
    }
    void pop(){
        qFront = (qFront + 1)%arrayLength;//弹出操作，即将 qFront 向前移动 1，模 arrayLength 确保 qFront 不超过
    }
private:
    T *element = nullptr;//元素数组指针
    int qBack;//记录队尾,随着元素入队而移动
    int qFront;//记录队首，随着元素出队而移动
    int arrayLength;//记录数组大小
};
//循环队列结束

struct Node{
    Node(){
        v = 0;
        next = nullptr;
    }
    Node(int value){
        v = value;
        next = nullptr;
    }
    int v;//v 代表点号
    Node* next;
};

class graph{
public:
    void push(Node* a, int b){//第 a 号节点与第 b 号节点之间建立联系
            Node* temp =    new Node(b);
            Node* last = a;
            Node* cnt = a->next;
            while(cnt != nullptr && cnt->v < b){
                last = cnt; cnt = cnt->next;
            }
            if(cnt == nullptr){
                last->next = temp;
            }
```

```
            else if(cnt->v == b) return;
            else {
                last->next = temp;
                temp->next = cnt;
            }
        }
    //有序插入操作完成
    void deleteElement(Node*cnt, int b){
        Node* last = cnt;
        cnt = cnt->next;
        if(cnt == nullptr)return;
        while(cnt != nullptr &&cnt->v != b){
            last = cnt;
            cnt = cnt->next;
        }
        last->next = cnt->next;
        delete cnt;
    }
    graph(){
        int m;
        scanf("%d%d%d%d",&n,&m,&s,&t);
        parent = new int[n + 1];
        for(int i = 0; i <= n; i++){
            parent[i] = i;
        }
        alist = new Node[n + 1];
        for(int i = 1; i<=n; i++)alist[i].v = i;
        for(int i = 1; i <= m; i++){
            int sym, u, v ;scanf("%d%d%d",&sym,&u,&v);
            if(sym == 0){
                push(&alist[u],v);
                push(&alist[v],u);
            }
            else if(sym == 1){
                deleteElement(&alist[u],v);
                deleteElement(&alist[v],u);
            }
        }
    }
    ~graph(){

    }
    int findacs(int son){//找到祖宗节点的同时更新祖宗节点
        if(son == parent[son]) return parent[son];
        parent[son] = findacs(parent[son]);
        return parent[son];
```

```
}
void merge(int a, int b){
    int acsa = findacs(a);
    int acsb = findacs(b);
    if(acsa == acsb) return;
    else if(acsa > acsb) parent[acsa] = acsb;//最后要多更新一次
    else parent[acsb] = acsa;//最后要多更新一次
}
void update(){
    for(int i = 1; i <= n; i++){//更新联通子图
        Node * temp = alist[i].next;
        while(temp != nullptr){
            merge(i,temp->v);
            temp = temp->next;
        }
    }//联通子图更新完毕,联通情况记录在了 parent 里
    for(int i = 1; i <= n; i++){
        findacs(i);
    }//更新完毕
}
void Dividenum(){
    int count = 0;
    for(int i = 1; i<=n; i++){
        if(i == parent[i])count++;
    }
    std::cout<<count<<'\n';//输出联通分量
    for(int i = 1; i<=n; i++){
        if(i == parent[i]) std::cout<<i<<' ';
    }
    std::cout<<'\n';
}
int dfs(int*&ans, bool *&sym, Node* cnt) {
    int sum = 0;sym[cnt->v] = 0;//置为 0 表示访问过
    ans[dfscount] = cnt->v;//将该点放入数组
    dfscount++;//数组 count++
    while(cnt != nullptr){
        if(sym[cnt->v] == 1){//如果没有被访问过
            sum+=dfs(ans, sym, &alist[cnt->v]);//递归查找下一个
        }
        cnt = cnt->next;
    }
    return sum;
}
void dfs(){//深度优先搜索
    dfscount = 0;
    int *ans = new int[n + 1];
```

```
        for(int i = 1; i <= n; i++)ans[i] = 0;
        bool*sym = new bool[n + 1];
        for(int i = 1; i <= n; i++) sym[i] = 1;
        dfs(ans, sym, &alist[s]);
        int count = 1;
        while(ans[count]!=0) count++;
        std::cout<<count<<'\n';
        for(int i = 0; i < count; i++)std::cout<<ans[i]<<' ';
        std::cout<<'\n';
        delete []ans; delete []sym;
        return;
    }
    void Out(Node* cnt){
        while(cnt!=nullptr){
            std::cout<<cnt->v<<' ';
            cnt = cnt->next;
        }
    }
    void bfs(){///广度优先搜索，在广度优先搜索中记录层数，从而找到 s 到 t 的最短路径
        int * ans = new int[n + 1];
        int count = 1;//ans 索引
        Queue<Node*>q(n+1);
        q.push(&alist[t]);
        bool *sym = new bool[n + 1];
        for(int i = 1; i <= n; i++) sym[i] = 1;
        sym[t] = 0;
        //轮次记录
        int symfind = 1;//找 t 点
          times = 0;//bfs 找到 t 点的遍历轮次
        int lasttimes = 1;//记录上层推入了几个点，也就是下一层要遍历的几个点
        int cnttimes = 0;//记录这层遍历了几个点， lasttimes 和 cnttimes 当 cnttimes == 0 时会进行
数值交换
        //轮次记录
        while(!q.empty()){//如果队列不为空,持续循环
            Node* temp = q.top();
            q.pop();
            ans[count] = temp->v;
            count++;
            if(symfind == 1 && cnttimes == 0){//如果没找到 t 且 cnttimes 为 0，则代表进入下一层
                cnttimes = lasttimes;
                lasttimes = 0;
                times++;
            }
            cnttimes--;
            while(temp!=nullptr){//遍历该点的周围所有点
                if(temp!=nullptr && sym[temp->v] == 1){//成功 push 进下一个点
```

```
                        if(temp->v == s) {//找到 t 点
                            symfind = 0;
                        }
                        if(symfind == 1){//没找到 t 点
                            lasttimes++;
                        }
                        q.push(&alist[temp->v]);//压入临近点
                        sym[temp->v] = 0;
                    }
                    temp = temp->next;
                }
            }
            std::cout<<count - 1<<'\n';
            for(int i = 1; i < count; i++){
                std::cout<<ans[i]<<' ';
            }
            std::cout<<'\n';
            delete []sym;
            delete []ans;
        }
        void OutRoad(){
            if(parent[s] != parent[t]){
                std::cout<<"-1"<<'\n';
            }
            else std::cout<<times<<'\n';
            return;
        }
private:
        int n;
        Node* alist;
        int s;
        int t;
        int dfscount;//深度优先数组索引
        int* parent;//记录并查集
        int times;//记录 bfs 层数
};
int main(){
        graph g;
        g.update();
        g.Dividenum();
        g.dfs();
        g.bfs();
        g.OutRoad();
        return 0 ;
}
```

